Upside Dine: Free Distributed Microservice Architecture

If your primary goal is to **learn and demonstrate a true microservice architecture with load balancing** completely for free, we can orchestrate a highly impressive, distributed setup using modern serverless databases and multiple free cloud accounts.

Here is the "Ultimate Free Architecture" that perfectly demonstrates load balancing, distributed state, and microservices:

🌐 1. The Frontend Edge

- **Service:** `Netlify`
- **What it does:** Hosts your React/Vite app on a global CDN.
- **Cost:** Free

🗄️ 2. The Distributed State (Databases)

For a load-balanced backend, the database and cache *must* be externalized (not stored on the same server as the API).

- **Primary Database (PostgreSQL):** `Supabase` (Provides an incredible free tier Postgres DB).
- **Cache & Message Broker (Redis):** `Upstash` (Provides a free serverless Redis database for Celery and WebSockets).
- **Cost:** Free

⚖️ 3. The Load Balanced API Layer (The Core Microservice)

To demonstrate load balancing, we will deploy identical clones of your Django Backend to different free providers, and use a Load Balancer to split the traffic between them.

- **Backend Node 1:** `Render` (Free Tier)
- **Backend Node 2:** `Koyeb` (Free Tier) or a second `Render` account.
- **The Load Balancer:** We can install **Nginx on your IITK Server** (or use Cloudflare) configured as a Round-Robin Load Balancer. It will receive all API requests from Netlify and distribute them 50/50 between Backend Node 1 and Node 2.
- **Cost:** Free

🧠 4. The Heavy Compute Node (ML Microservice)

- **Service:** Your `IITK Server`
- **What it does:** Runs the YOLOv8 `ml_service` via Docker. Since the IITK server has good compute, it handles the video processing flawlessly. It reads instructions from Upstash Redis and sends results back to the Load Balancer.
- **Cost:** Free (University provided)

⚙️ 5. Asynchronous Workers

- **Service:** Your `IITK Server` (or Koyeb)
- **What it does:** Runs the `celery_worker` and `celery_beat`. They connect to the Upstash Redis queue and Supabase DB to process background tasks without slowing down your HTTP APIs.

Why this architecture is impressive:

1. **True Load Balancing:** API traffic is distributed across two different cloud providers. If Render goes down, Koyeb keeps the app alive.
2. **Stateless APIs:** Because Postgres and Redis are externalized (Supabase/Upstash), your Django backend containers are completely stateless—a core requirement of microservices.
3. **Compute Segregation:** Heavy AI workloads (YOLO) are isolated to dedicated hardware (IITK) so they don't crash your web APIs.

Let me know if this sounds like the architecture you want to build! If yes, our first step will be setting up **Supabase** and **Upstash** so we can decouple your database.